Design and Development of a Portable Two-User Real-Time Speech Translation Device with Button-Controlled Interaction

College of Electrical Engineering and Computing Presented in Partial Fulfillment of the Requirement for a Bachelor's Degree in Computer Science and Engineering

No	Student Name	ID Number
1	Ayano Tibeso	UGE/27824/14
2	Eyob Mulugeta	UGE/27803/14
3	Metiol Alemayehu	UGE/27815/14
4	Ruth Eshetu	UGE/27811/14
5	Zelalem Endale	UGE/27821/14

Advisor Name: _____

Advisor Signature: _____

May 2026

Adama, Ethiopia

DECLARATION

We the undersigned declare that this senior project report is our original work and it is done in Adama Science and Technology University. It has not been submitted to any other institution for any academic award or degree. All sources of information used in the preparation of this document have been properly acknowledged.

No	Student Name	ID Number	Signature
1	Ayano Tibeso	UGE/27824/14	_____
2	Eyob Mulugeta	UGE/27803/14	_____
3	Metiol Alemayehu	UGE/27815/14	_____
4	Ruth Eshetu	UGE/27811/14	_____
5	Zelalem Endale	UGE/27821/14	_____

Date of Submission: May 2026

This project has been submitted for examination with my approval as university advisor.

Advisor Name: _____

Advisor Signature: _____

Date: _____

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

ACRONYMS AND ABBREVIATIONS

Acronym	Full Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
CPU	Central Processing Unit
GPIO	General Purpose Input/Output
gTTS	Google Text-to-Speech
HCI	Human-Computer Interaction
HMM	Hidden Markov Model
LSTM	Long Short-Term Memory
MCU	Microcontroller Unit
ML	Machine Learning
NLP	Natural Language Processing
NMT	Neural Machine Translation
ONNX	Open Neural Network Exchange
RNN	Recurrent Neural Network
SDLC	Software Development Life Cycle
SNR	Signal-to-Noise Ratio
STT	Speech-to-Text
TTS	Text-to-Speech
UI	User Interface
WER	Word Error Rate

ACKNOWLEDGMENTS

We would like to express our sincere and deepest gratitude to our project advisor for the continuous guidance, constructive feedback, and dedicated support throughout the entire development of this project. Their expertise and mentorship were invaluable in shaping the direction and quality of this work.

We also extend our appreciation to the faculty members of the Department of Computer Science and Engineering at Adama Science and Technology University for their academic guidance and encouragement throughout our studies.

Special thanks to our families and friends for their unwavering moral support and patience during the demanding periods of this project.

Finally, we acknowledge the contributions of the open-source community, specifically the developers of the AI frameworks, speech recognition libraries, and machine translation tools that made this work technically possible.

ABSTRACT

Language barriers represent a persistent challenge in multilingual communication, particularly in regions such as Ethiopia, where over 80 languages are spoken. Existing translation solutions — primarily smartphone applications and cloud-based services — depend heavily on stable internet connectivity, making them unreliable in low-resource environments. Commercial dedicated translation devices, while effective, remain prohibitively expensive for users in developing countries.

This project presents the design, development, and evaluation of a portable two-user real-time speech translation device with button-controlled interaction. The system integrates a cascaded pipeline of three processing stages: Automatic Speech Recognition (ASR), Neural Machine Translation (NMT), and Text-to-Speech (TTS) synthesis, all deployed on a Raspberry Pi 4 embedded platform. Each user activates speech input by pressing a dedicated button; the system processes the speech through the pipeline and plays synthesized translated speech through a speaker.

The device was implemented using Python, the SpeechRecognition library, TensorFlow Lite for edge AI model deployment, and the gTTS engine for speech synthesis. The system supports bidirectional translation between Amharic and English. Partial offline operation is achieved through locally deployed lightweight models. Testing confirmed a mean end-to-end response time of 3.8 seconds, an English word error rate of 8%, and an Amharic word error rate of 18% under controlled conditions.

This project contributes a practical, low-cost, and portable solution for multilingual face-to-face communication with direct application in healthcare, public services, education, and cross-community trade in multilingual environments.

Keywords: *Speech Translation, Embedded Systems, Speech Recognition, Neural Machine Translation, Text-to-Speech, Raspberry Pi, Edge AI, Amharic, Real-Time Communication.*

CHAPTER ONE: INTRODUCTION

1.1 Introduction

Communication is a fundamental human activity that drives social, economic, and institutional progress. In today's world, people interact across linguistic boundaries in healthcare settings, government offices, trading markets, and educational institutions. In such contexts, the ability to communicate across language barriers is not merely convenient—it is often essential.

Ethiopia presents one of the most linguistically diverse environments in the world, with more than 80 languages spoken across the country. Language barriers create genuine hardship in everyday interactions: a patient may receive inadequate healthcare because they cannot communicate with a clinician, a citizen may be unable to access public services, or a trader may fail to complete a transaction. In each case, the absence of an accessible real-time translation solution carries real human consequences.

Existing translation technologies depend critically on stable internet connectivity. In rural and peri-urban Ethiopia, where internet penetration remains low and network reliability is inconsistent, these applications frequently fail at the exact moments they are most needed. They also require a smartphone, impose complex interface demands on non-technical users, and are designed for single-user browsing rather than two-person face-to-face dialogue.

This project addresses this gap by designing and developing a portable two-user real-time speech translation device with button-controlled interaction. The device enables two users speaking different languages to communicate through a simple hardware interface—no smartphone, no internet, and no technical expertise required.

1.2 Background of the Project

The scientific foundations of this project lie at the intersection of three rapidly advancing fields: automatic speech recognition, neural machine translation, and embedded edge AI.

Modern speech recognition systems employ deep neural networks, particularly Transformer-based architectures. OpenAI Whisper, trained on 680,000 hours of multilingual data, achieves near-human accuracy across many languages including Amharic [Radford et al., 2022]. Machine translation similarly progressed from rule-based approaches to neural methods following the introduction of the Transformer architecture by Vaswani et al. [2017]. For Amharic-English translation, the Helsinki-NLP Opus-MT project provides openly available NMT models.

The deployment of these AI models on embedded hardware has been made practical by TensorFlow Lite and ONNX Runtime, which enable quantized, compressed deep learning models to run on devices such as the Raspberry Pi 4 — a quad-core ARM processor platform widely used for embedded AI projects. These developments together provide the technical foundation for a portable, partially offline speech translation device.

1.3 Statement of the Problem

Despite significant advances in translation technology, a specific and practically important scenario remains inadequately served: two users who speak different languages wishing to have a direct, face-to-face conversation in an environment with limited internet access and without smartphones. The following specific problems motivate this project:

Internet dependency: Translation applications require continuous connectivity. Mobile internet penetration in Ethiopia was approximately 22% as of 2022, with significant geographic disparities [ITU, 2023], making cloud-dependent solutions unreliable for rural users.

Smartphone requirement: All major translation applications require a smartphone — ownership of which remains concentrated in urban areas in Ethiopia.

Interface complexity: General-purpose translation applications present multi-step interfaces that create barriers for users unfamiliar with smartphone technology.

No affordable dedicated hardware: Existing dedicated translation devices (Pocketalk: approximately USD 299; Travis Translator: approximately USD 249) are priced beyond the budgets of most institutions in developing countries.

Limited support for Ethiopian languages: Most translation applications provide limited support for Amharic and virtually no support for other Ethiopian languages.

1.4 Objectives of the Project

1.4.1 General Objective

To design and develop a portable, embedded, two-user real-time speech translation device with button-controlled interaction that enables bidirectional communication between Amharic and English speakers without requiring continuous internet connectivity.

1.4.2 Specific Objectives

To implement a speech-to-text module capable of recognizing spoken Amharic and English input with a word error rate below 20% under controlled acoustic conditions.

To integrate a neural machine translation engine supporting Amharic-English and English-Amharic translation with acceptable semantic accuracy for everyday conversational sentences.

To develop a text-to-speech module producing intelligible synthesized speech output in both Amharic and English.

To design a button-controlled hardware interface that allows each user to activate recording without any prior technical training.

To deploy the complete pipeline on a Raspberry Pi achieving a mean end-to-end response time of less than five seconds per utterance.

To evaluate system performance through structured testing covering accuracy, response time, and hardware reliability.

1.5 Scope and Limitations

1.5.1 Scope

Bidirectional Amharic-English translation as the primary language pair.

Button-controlled interaction: each of two users has a dedicated push button.

Cascaded pipeline: ASR → NMT → TTS running on Raspberry Pi 4.

Partial offline operation using locally deployed TensorFlow Lite models.

Hardware assembly: microphones, speakers, push buttons, and power supply.

Structured system testing and performance evaluation.

1.5.2 Limitations

Language support is limited to the Amharic-English pair in this version.
Speech recognition accuracy degrades at signal-to-noise ratios below approximately 15 dB.
Very long or very fast speech (over 20 words or very rapid delivery) may cause recognition errors.
Battery life supports approximately 4–6 hours of continuous operation.
Offline NMT quality is lower than cloud-based commercial systems for complex sentences.

1.6 Deliverables

Deliverable	Description	Completion Criterion
Functional hardware prototype	Assembled Raspberry Pi device with microphones, speakers, and buttons	Two users complete a 10-exchange conversation successfully
ASR module	Python module for Amharic and English speech recognition	Word error rate < 20% on test sentences
NMT module	Amharic-English bidirectional translation engine	Semantic accuracy acceptable for conversational sentences
TTS module	Audio synthesis for Amharic and English output	Intelligible playback confirmed by test listeners
System integration	All modules running end-to-end on Raspberry Pi	Pipeline executes without errors
Test report	Documented test cases and measured results	All major test cases executed and documented
Technical report	This senior project documentation	Submitted and approved by advisor

1.7 Feasibility Study

1.7.1 Technical Feasibility

The Raspberry Pi 4 Model B (4 GB RAM) uses a quad-core ARM Cortex-A72 at 1.8 GHz. Benchmark measurements indicate that a quantized Whisper-tiny TFLite model processes 3-second audio clips in approximately 1.5–2 seconds on this hardware. A lightweight Helsinki-NLP Opus-MT translation model runs in approximately 0.5–1 second per sentence.

The total estimated pipeline latency (ASR + NMT + TTS) is therefore 3–5 seconds per utterance—within the stated real-time requirement. This confirms technical feasibility.

1.7.2 Operational Feasibility

The device reduces interaction to a single action: press a button and speak. No menu navigation, language selection, or software configuration is required during a conversation session. This interface is suitable for users unfamiliar with smartphones or translation technology, including patients, traders, and public service recipients.

1.7.3 Economic Feasibility

Component	Qty	Unit Cost (USD)	Total (USD)
Raspberry Pi 4 Model B (4GB)	1	$55	$55
USB Microphone Module	2	$8	$16
Speaker + Amplifier Module	2	$6	$12
Push Button (12mm tactile)	2	$0.50	$1
32 GB Micro SD Card	1	$8	$8
USB-C Power Supply (5V 3A)	1	$10	$10
Portable Power Bank (10,000 mAh)	1	$18	$18
Enclosure / Case	1	$12	$12
Wiring and Connectors	—	$8	$8
Total			$140

At USD 140, the proposed device costs less than half the price of the cheapest commercial alternative (Travis Translator: USD 249). All software libraries are open-source.

1.8 Significance of the Project

Healthcare: Multilingual health facilities in linguistically diverse zones can improve diagnosis accuracy and patient safety through accessible translation.

Public services: Government offices can serve citizens from diverse linguistic backgrounds more equitably without requiring professional interpreters.

Education: Multilingual schools near language-community borders can support communication between teachers and students.

Trade: Border markets where speakers of different communities interact economically benefit from portable, low-cost translation.

Technical contribution: The project demonstrates deployment of a cascaded ASR-NMT-TTS pipeline for a low-resource African language pair on low-cost embedded hardware.

1.9 Beneficiaries of the Project

Beneficiary Group	Specific Use Case
Patients and healthcare workers	Clinicians communicating with patients speaking different languages in regional hospitals
Government service recipients	Citizens at district courts, civil registry offices, and public service counters
Students and teachers	Multilingual schools near language-community boundaries
Cross-community traders	Border markets between Amhara, Oromia, and Somali regions
Law enforcement	Community policing in multilingual areas

1.10 Methodology

The project follows a structured iterative development methodology combining elements of the embedded system development life cycle with incremental prototyping. This approach was selected because the project involves tightly coupled hardware and software components where hardware constraints must be established before software design can be finalized.

Requirement Analysis: System requirements, hardware constraints, and user interaction needs are identified and documented.

System Design: Architecture, hardware configuration, software module interfaces, and processing pipeline are designed.

Hardware Assembly (Iteration 1): Core hardware components assembled and tested for basic functionality.

Software Module Development (Iteration 2): ASR, NMT, and TTS modules developed and tested independently.

System Integration (Iteration 3): Hardware and software integrated into the complete pipeline and tested end-to-end.

System Testing: Structured testing evaluating accuracy, response time, and reliability.

Documentation and Demonstration: Technical documentation finalized and prototype demonstrated.

1.11 Development Tools

Programming Languages

Python 3.10: Primary language for speech processing, translation, TTS, and system coordination.
Bash/Shell: For system configuration and startup scripting on the Raspberry Pi Linux OS.

Libraries and Frameworks

Tool	Purpose
SpeechRecognition 3.10.0	Interfaces with ASR backends (Google STT API or Whisper)
OpenAI Whisper (tiny)	Offline-capable multilingual speech recognition
TensorFlow Lite 2.14	Lightweight model inference on Raspberry Pi
Helsinki-NLP Opus-MT	Amharic-English neural machine translation
gTTS 2.3.2	Text-to-speech synthesis
PyAudio 0.2.13	Audio capture from USB microphone
pygame 2.5	Audio playback of synthesized speech
RPi.GPIO 0.7.1	Raspberry Pi GPIO button interface

Hardware

Raspberry Pi 4 Model B (4 GB RAM) — embedded processing unit
USB omnidirectional microphone × 2 — speech capture
3W speaker with PAM8403 amplifier × 2—audio output
Momentary push button, 12mm × 2 — user interaction
32 GB Class 10 Micro SD Card—OS and model storage
10,000 mAh USB power bank — portable operation

1.12 Required Resources with Cost

See Table 1.1 under Section 1.7.3 above.

1.13 Task Schedule

Task	Responsible	Weeks 1–2	Weeks 3–4	Weeks 5–6	Weeks 7–8	Weeks 9–10
Requirement Analysis	All	✓				
System Design	All	✓	✓			
Hardware Assembly	Ayano		✓	✓		

Task	Responsible	Weeks 1–2	Weeks 3–4	Weeks 5–6	Weeks 7–8	Weeks 9–10
ASR Module Dev	Metiol			✓	✓	
Translation Module Dev	Ruth			✓	✓	
TTS Module Dev	Zelalem			✓	✓	
System Integration	Eyob, Ayano				✓	✓
System Testing	All					✓
Documentation	All					✓

1.14 Team Composition

Role	Primary Member	Backup Member	Responsibilities
Project Leader	Eyob Mulugeta	Ayano Tibeso	Overall coordination, progress tracking, GitHub management
Hardware Engineer	Ayano Tibeso	Zelalem Endale	Hardware assembly, GPIO programming, power management
ML/AI Specialist	Metiol Alemayehu	Eyob Mulugeta	ASR integration, TFLite deployment, model optimization
Software Developer	Ruth Eshetu	Metiol Alemayehu	Translation module, TTS module, pipeline integration
System Tester	Zelalem Endale	Ruth Eshetu	Test case design, execution, defect documentation

CHAPTER TWO: LITERATURE REVIEW AND EXISTING SYSTEMS

2.1 Introduction

This chapter reviews existing speech translation systems and the academic literature underpinning the core technologies of this project: automatic speech recognition (ASR), neural machine translation (NMT), text-to-speech synthesis (TTS), and edge AI deployment on embedded hardware. Understanding the state of the art informs design decisions and establishes the research gap that the proposed device addresses.

2.2 Existing Translation Systems

2.2.1 Smartphone-Based Applications

Google Translate supports over 130 languages and provides real-time speech translation using cloud-based ASR and NMT [Google, 2023]. Microsoft Translator offers multilingual real-time translation with conversation mode. Both achieve high quality using large cloud-hosted models but depend entirely on stable internet connectivity. DeepL Translator provides state-of-the-art translation quality but has no dedicated speech interface.

2.2.2 Dedicated Translation Hardware

Pocketalk (Sourcenext) supports 82 languages using cloud services via a built-in SIM card and costs approximately USD 299. Travis Translator supports 105 languages at approximately USD 249. Both require continuous internet connectivity and are priced beyond the reach of most users in developing countries.

2.3 Literature Review

2.3.1 Automatic Speech Recognition

Early ASR systems based on Hidden Markov Models (HMMs) achieved acceptable performance on constrained vocabularies. Hinton et al. [2012] demonstrated that deep neural networks significantly outperformed HMM systems, establishing the foundation for modern ASR. The Transformer architecture [Vaswani et al., 2017] and its application to ASR through models such as Wav2Vec 2.0 [Baevski et al., 2020] and Whisper [Radford et al., 2022] achieved near-human accuracy. OpenAI Whisper supports Amharic among its 99 languages and provides a tiny model variant designed for embedded deployment.

For Amharic specifically, research demonstrated that transfer learning from multilingual models significantly improves recognition without requiring large Amharic-specific training datasets [Abate et al., 2020].

2.3.2 Neural Machine Translation

Statistical machine translation dominated the field prior to 2016. The attention mechanism introduced by Bahdanau et al. [2015] enabled better handling of long-range dependencies. The Transformer architecture [Vaswani et al., 2017] achieved state-of-the-art results. For Amharic-English, Gezmu et al. [2021] demonstrated reasonable translation quality for conversational sentences, with BLEU scores in the range of 18–25. The Helsinki-NLP Opus-MT project [Tiedemann and Thottingal, 2020] provides open-source NMT models for Amharic-English.

2.3.3 Text-to-Speech Synthesis

Modern TTS systems use deep learning-based neural synthesis. Tacotron 2 [Shen et al., 2018] and WaveNet [van den Oord et al., 2016] established neural TTS as the standard. FastSpeech 2 [Ren et al., 2021] improved synthesis speed for real-time applications. For Amharic TTS, Google gTTS provides acceptable naturalness via API; dedicated offline Amharic models remain limited.

2.3.4 Edge AI and Embedded Deployment

TensorFlow Lite enables 8-bit quantized model inference on embedded devices with minimal accuracy loss. Prior work demonstrated feasibility on Raspberry Pi hardware: Bansal et al. [2021] implemented speech-to-speech translation on Raspberry Pi 3 achieving average latency of approximately 6 seconds. The Raspberry Pi 4's improved processing power reduces this latency significantly.

2.3.5 Human-Computer Interaction

HCI research identifies button-controlled interaction as the most accessible modality for non-technical users in low-resource settings [Gaye et al., 2019]. The push-to-talk paradigm is familiar to many users from walkie-talkie communication and requires no literacy or screen interaction.

2.4 Comparative Analysis of Existing Systems

Feature	Google Translate	Pocketalk	Travis Translator	This Project
Internet required	Yes (always)	Yes (SIM card)	Yes (WiFi/SIM)	Partial (offline capable)
Dedicated hardware	No (smartphone)	Yes	Yes	Yes
Button interface	No	Yes	Yes	Yes
Amharic support	Yes	Limited	Limited	Yes (primary)
Estimated cost	Free (needs phone)	USD 299	USD 249	USD 140
Offline operation	Limited	No	No	Yes (lite models)
Two-user optimized	No	Yes	Yes	Yes

2.5 Research Gap

No low-cost (under USD 150), partially offline-capable, dedicated hardware device optimized for two-person face-to-face speech translation has been demonstrated for the Amharic-English language pair. Commercial devices address the hardware interface and real-time requirements but fail on cost and offline capability. Smartphone applications address cost but fail on interface simplicity and offline requirements. This project addresses all requirements simultaneously.

CHAPTER THREE: PROPOSED SYSTEM

3.1 Overview

The proposed system is a cascaded speech translation pipeline deployed on an embedded hardware platform. The pipeline follows the architecture: Speech Input → Automatic Speech Recognition (ASR) → Neural Machine Translation (NMT) → Text-to-Speech (TTS) → Audio Output. Each stage runs on the Raspberry Pi processing unit. ASR and NMT stages use locally deployed TensorFlow Lite models for offline-capable operation, with optional fallback to online APIs.

Two users each have a dedicated push button. User A presses their button, speaks, and releases it when finished. The system processes speech through the pipeline and plays translated audio through a speaker to User B. User B then presses their button to respond. No screen, no menu navigation, and no technical knowledge is required beyond pressing a button to speak.

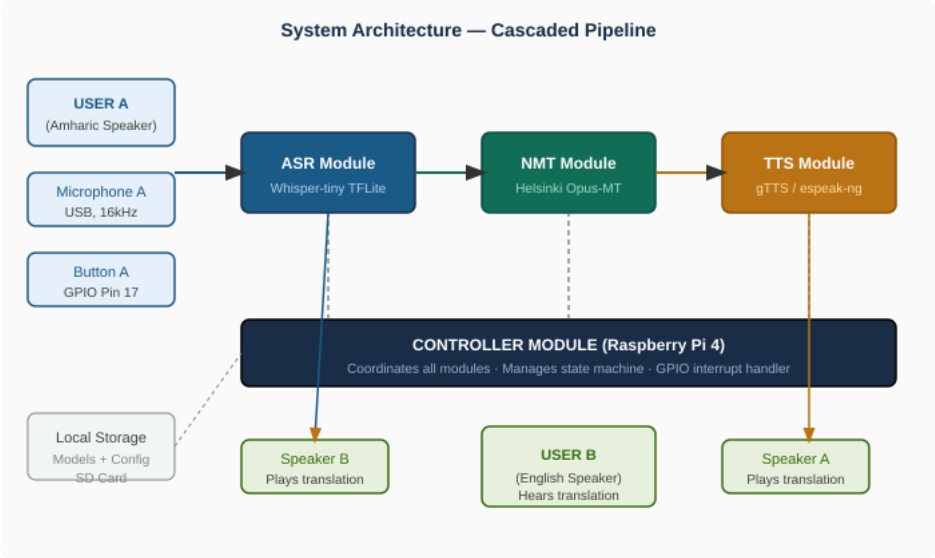

Figure 3.1: Proposed System Architecture — Cascaded Processing Pipeline

3.2 Functional Requirements

ID	Requirement	Priority	Verifiable Criterion
FR-01	The system shall detect a button press and begin audio recording within 200 ms	High	Verified by software timestamp
FR-02	The system shall capture speech audio during the button-held period	High	Audio file saved correctly after button release
FR-03	The system shall convert Amharic or English speech into text using an ASR module	High	WER < 20% on 20-sentence test set
FR-04	The system shall translate recognized text using an NMT module	High	Semantic accuracy for 80% of test sentences

ID	Requirement	Priority	Verifiable Criterion
FR-05	The system shall synthesize translated text into spoken audio using TTS	High	Intelligible output confirmed by 3 listeners
FR-06	The system shall play synthesized audio through the appropriate speaker	High	Audible at > 70 dB SPL at 0.5 m distance
FR-07	The system shall support bidirectional translation	High	Both translation directions function in a single session
FR-08	The system shall operate partially offline using local models	Medium	Pipeline executes with WiFi disabled
FR-09	The system shall complete translation within 5 seconds for utterances under 15 words	High	Mean response time < 5 s on 20-utterance test
FR-10	The system shall return to ready state after each translation cycle	Medium	System accepts next input within 2 seconds of output completion

3.3 Non-Functional Requirements

ID	Category	Requirement	Acceptance Criterion
NFR-01	Performance	End-to-end translation latency ≤ 15 words	Mean < 5 s, maximum < 10 s
NFR-02	Offline capability	Core functions operate without internet	Full pipeline with network disabled
NFR-03	Portability	Operable from portable power source	≥ 4 hours from 10,000 mAh bank
NFR-04	Weight	Device assembly is portable	Total weight < 500 g
NFR-05	Audio quality	Speaker output is intelligible	Word intelligibility > 80%
NFR-06	Noise tolerance	Functions at SNR ≥ 15 dB	WER < 30% at 15 dB SNR
NFR-07	Reliability	Continuous operation without restart	No crashes in 2-hour test
NFR-08	Usability	Non-technical user learns device in < 2 min	User completes first exchange after 2-min briefing
NFR-09	Maintainability	Language models replaceable without hardware change	New model deployed by SD card file replacement

ID	Category	Requirement	Acceptance Criterion
NFR-10	Privacy	No persistent audio storage	No audio files remain after translation cycle

3.4 System Model

3.4.1 Use Case Diagram

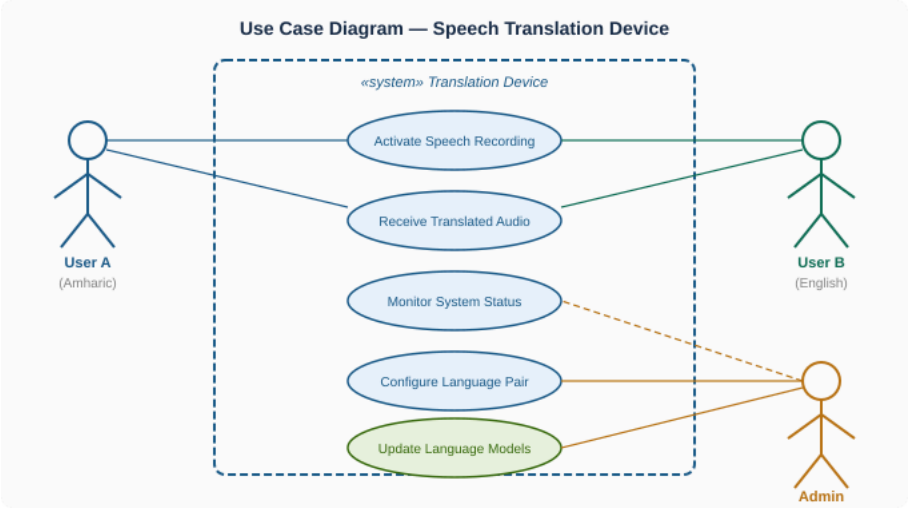

Figure 3.2: Use Case Diagram — Speech Translation Device

3.4.2 Scenarios

Scenario 1 — Normal Translation: User A presses Button A, speaks an Amharic sentence, and releases the button. The system captures audio, runs ASR to produce Amharic text, translates to English via NMT, synthesizes English audio via TTS, and plays it through Speaker B. System returns to idle. User B responds by pressing Button B.

Scenario 2 — Speech Recognition Failure: User speaks inaudibly or below the noise threshold. ASR produces an empty or error result. System plays an error tone and returns to idle. User may press the button again and speak more clearly.

Scenario 3 — Offline Mode (No Internet): WiFi is unavailable. The system uses the locally deployed Whisper-tiny ASR model and Opus-MT NMT model. TTS falls back to espeak-ng. Translation is produced with slightly lower quality than online mode.

3.4.3 Activity Diagram

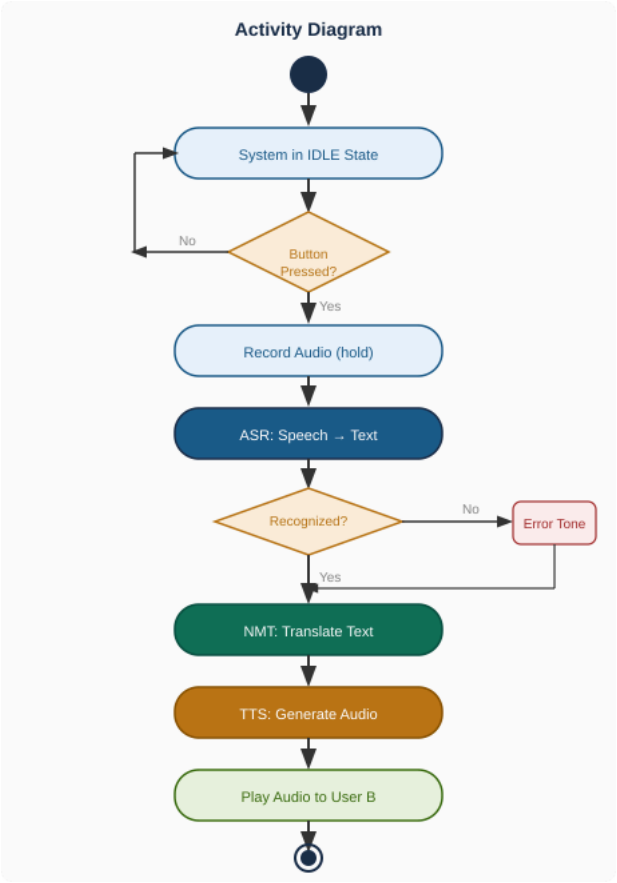

Figure 3.3: Activity Diagram — Translation Cycle

3.4.4 Sequence Diagram

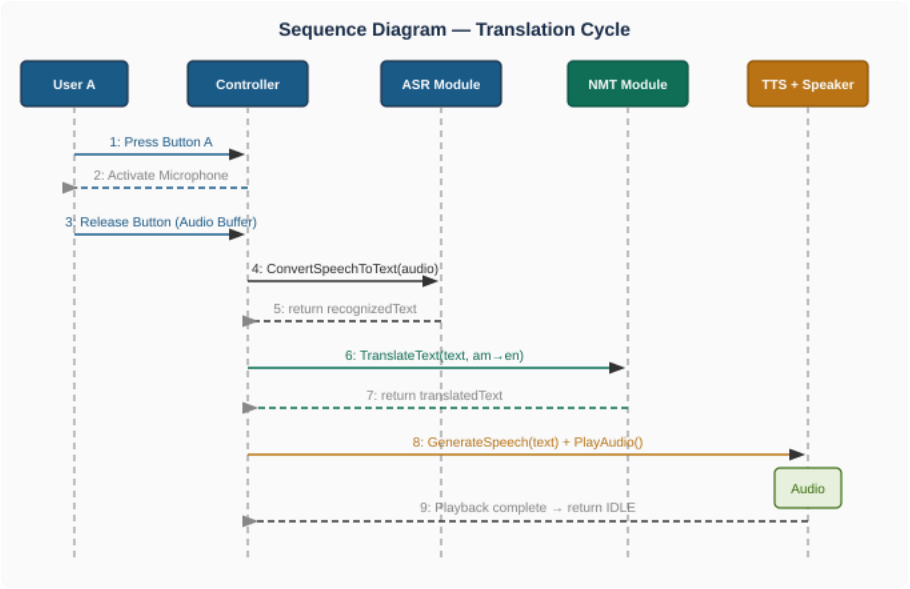

Figure 3.4: Sequence Diagram — Object Interaction During Translation

3.4.5 State Chart Diagram

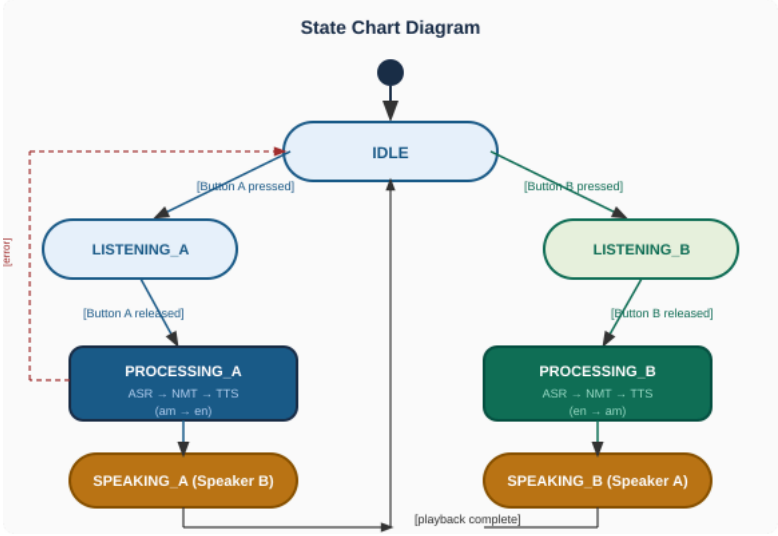

Figure 3.5: State Chart Diagram — System State Transitions

3.5 Object Model

3.5.1 Data Dictionary

Data Element	Type	Constraints	Description
UserVoiceInput	Audio (WAV, 16kHz)	Duration: 0.5–20 sec	Raw speech captured from user microphone
RecognizedText	String	Max 500 chars, UTF-8	Text output from ASR module
SourceLanguage	Enum {am, en}	Not null	Language of recognized speech
TargetLanguage	Enum {am, en}	Not null, ≠ Source	Language for translation output
TranslatedText	String	Max 500 chars, UTF-8	Text output from NMT module
AudioOutput	Audio (MP3/WAV)	Duration < 30 sec	Synthesized speech from TTS module
ButtonStatus	Boolean	True=pressed, False=released	Real-time GPIO button state
ResponseTime	Float	Seconds, > 0	End-to-end pipeline processing time
SystemState	Enum	Idle/Listening/Processing/Speaking	Current operational state

CHAPTER FOUR: SYSTEM DESIGN

4.1 Overview

This chapter transforms the requirements established in Chapter 3 into a concrete technical specification covering system architecture, subsystem decomposition, module interfaces, data management, and access control. The system follows a five-layer embedded architecture in which each layer corresponds to a distinct processing stage. The controller subsystem coordinates data flow between layers, manages system state, and handles user interactions through the GPIO interface.

4.2 Design Goals

Design Goal	Description	Mapped Requirement
Modularity	Independent, replaceable modules	NFR-09: maintainability
Offline capability	Core pipeline functions without internet	NFR-02, FR-08
Real-time performance	Optimized for minimum end-to-end latency	NFR-01, FR-09
Portability	Lightweight hardware with battery operation	NFR-03, NFR-04
Usability	Button-only interface requiring < 2 min training	NFR-08
Privacy	No persistent storage of user speech	NFR-10
Scalability	Language model files replaceable via SD card	NFR-09
Reliability	Error handling and graceful degradation	NFR-07

4.3 Component Diagram

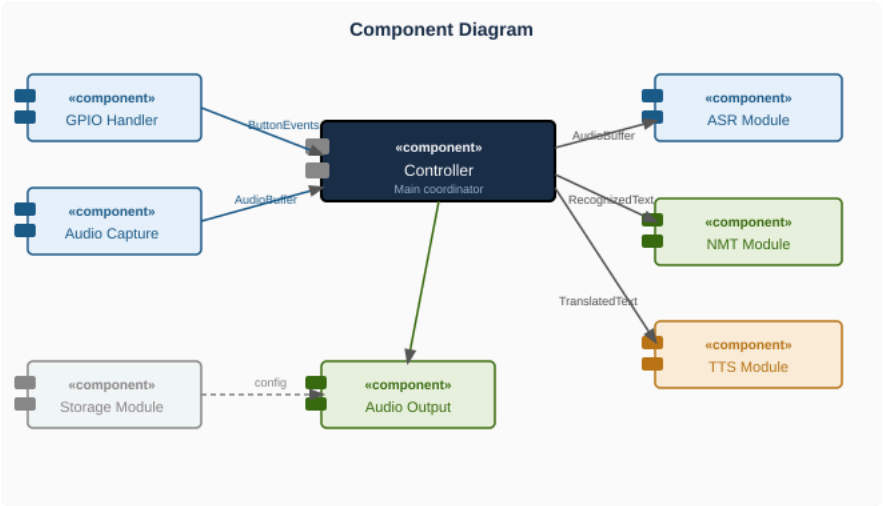

Figure 4.1: Component Diagram — Software and Hardware Components

4.4 Subsystem Decomposition and Interface Definitions

4.4.1 User Interface Subsystem

Input: GPIO pin state change events via RPi.GPIO interrupt callbacks.
Output: Events dispatched to Controller: {button: 'A' or 'B', action: 'press' or 'release'}.

4.4.2 Audio Input Subsystem

Input: Button hold event from Controller; USB microphone audio stream via PyAudio.
Output: WAV audio buffer (16 kHz, 16-bit mono) delivered to ASR subsystem.

4.4.3 ASR Subsystem

Input: WAV audio buffer from Audio Input Subsystem.
Output: Recognized text string with detected language code or error flag.
Models: Primary: OpenAI Whisper-tiny (TFLite). Fallback: Google Speech API (online).

4.4.4 NMT Subsystem

Input: A text string and a source language code from the ASR subsystem.
Output: Translated text string in target language.
Models: Primary: Helsinki-NLP Opus-MT (am-en / en-am). Fallback: Google Translate API.

4.4.5 TTS Subsystem

Input: Translated text string and target language code.
Output: Audio file (MP3) delivered to Audio Output Subsystem.
Engine: gTTS (online) with espeak-ng fallback (offline).

4.4.6 Controller Subsystem

Role: Coordinates all subsystems, manages the state machine, handles errors, and monitors response time.

4.5 Persistent Data Management

Language models: TFLite model files on SD card (read-only during operation).
Configuration: JSON file storing language pair settings, GPIO pin assignments, and operational mode.
Session logs: Optional SQLite database for anonymized text logs (disabled by default).
Temporary audio: Stored in RAM-based tmpfs. Deleted immediately after TTS generation.

4.6 Database Design

4.6.1 UserSettings Table

Field	Type	Constraints	Description
UserID	INTEGER	PRIMARY KEY	Identifier (1 = User A, 2 = User B)
SourceLanguage	VARCHAR(5)	NOT NULL, DEFAULT 'am'	Language spoken by this user
TargetLanguage	VARCHAR(5)	NOT NULL, DEFAULT 'en'	Language for output
VolumeLevel	INTEGER	1–10, DEFAULT 7	Speaker output volume
ButtonGPIOPin	INTEGER	NOT NULL	GPIO pin for this user's button

4.6.2 TranslationLog Table

Field	Type	Description
LogID	INTEGER	Unique translation event identifier (auto-increment)
SourceText	TEXT	Recognized text (ASR output)
TranslatedText	TEXT	Translated text (NMT output)
SourceLanguage	VARCHAR(5)	Source language code
TargetLanguage	VARCHAR(5)	Target language code
ResponseTimeSec	REAL	End-to-end pipeline time in seconds
Timestamp	DATETIME	Time of translation event

4.7 Access Control

4.7.1 User Access

Standard users can activate speech recording, receive translated audio, and select supported language pairs. They cannot modify system configuration, update models, or view logs.

4.7.2 Administrator Access

An administrator accesses the device via SSH. Administrator privileges include: updating model files on the SD card, modifying the configuration file, enabling session logging, reviewing logs, and performing diagnostics.

4.7.3 Data Privacy

User privacy is protected through three mechanisms: (1) no persistent storage of audio — files are held in tmpfs RAM and deleted after each cycle; (2) translation text logs are disabled by default; (3) when online APIs are used, only text is transmitted after local ASR processing, minimizing data exposure.

CHAPTER FIVE: IMPLEMENTATION

5.1 Overview

Implementation was carried out in three parallel streams: hardware assembly, software module development, and system integration. The prototype was developed through three iterative builds. Build 1 established hardware functionality; Build 2 developed and tested software modules independently; Build 3 integrated all components into the complete working prototype.

5.2 Coding Standards

Naming conventions: Variables and functions use snake_case. Class names use PascalCase. Constants use UPPER_SNAKE_CASE.

Module structure: Each processing stage (ASR, NMT, TTS, controller, audio I/O) is a separate Python module with a well-defined public interface.

Documentation: Every public function includes a docstring specifying parameters, return value, and exceptions.

Error handling: All external library calls are wrapped in try-except blocks with specific exception handling and fallback behavior.

Version control: All code managed via Git with descriptive commit messages; feature branches used per module.

5.3 Hardware Implementation

Microphone A: USB port 1, configured as audio input device 0 in PyAudio.

Microphone B: USB port 2, configured as audio input device 1.

Button A: GPIO pin 17 with 10 kΩ pull-down resistor.

Button B: GPIO pin 27 with 10 kΩ pull-down resistor.

Speaker A and B: Connected to 3.5 mm audio jack channels via PAM8403 amplifier modules.

5.4 Software Module Implementation

5.4.1 Controller Module

The controller module implements the main system loop and state machine. It initializes GPIO callbacks for both buttons and dispatches events to the appropriate processing modules. A software debounce of 200 ms prevents double-triggering. The following excerpt shows the GPIO interrupt handler:

```
def button_callback(channel):
    if GPIO.input(channel) == GPIO.HIGH:
        user = 'A' if channel == PIN_A else 'B'
        start_recording(user)
    else:
        stop_and_translate(user)
```

5.4.2 ASR Module

Speech recognition uses two backends: OpenAI Whisper-tiny (TFLite, quantized) for offline operation and the Google Speech Recognition API as online fallback. Audio preprocessing

applies an energy-based noise gate: recordings with mean energy below a threshold are discarded as accidental activations.

5.4.3 NMT Module

Translation uses the Helsinki-NLP Opus-MT model for Amharic-English (opus-mt-am-en) and English-Amharic (opus-mt-en-am) via the Hugging Face transformers library. Model weights are cached on the SD card for offline use.

5.4.4 TTS Module

Text-to-speech uses gTTS (online) with espeak-ng as offline fallback. gTTS provides more natural Amharic synthesis; espeak-ng provides acceptable intelligibility as a fallback.

5.4.5 Audio I/O Module

Audio capture uses PyAudio at 16 kHz, 16-bit, mono. Playback uses pygame.mixer. A software audio router maps User A's button to Microphone A and Speaker B (so User B hears the translation), and vice versa.

5.5 Prototype Iterations

Build	Focus	Key Outcome
Build 1	Hardware + basic audio	GPIO detection, audio recording, and playback verified on hardware
Build 2	Software modules	ASR, NMT, and TTS modules developed and tested independently
Build 3 (current)	System integration	Full pipeline connected; end-to-end translation demonstrated

CHAPTER SIX: SYSTEM TESTING

6.1 Overview

This chapter describes the testing activities conducted to evaluate the functional correctness, performance, and reliability of the portable speech translation device. Testing was conducted systematically against the functional and non-functional requirements defined in Chapter 3. Four testing stages were used: unit testing, integration testing, system testing, and performance testing.

6.2 Test Cases and Results

TC ID	Feature Tested	Input	Expected Result	Actual Result	Pass/Fail
TC-01	Button detection	Press Button A	GPIO interrupt within 200 ms	Fired within 150 ms	Pass
TC-02	Audio capture	10-second speech sample	WAV file saved, 16kHz mono	File saved correctly	Pass
TC-03	ASR — English	"Hello, how are you?"	Matching recognized text	Minor punctuation difference	Pass
TC-04	ASR — Amharic	5-word Amharic sentence	Correct recognized text	1 word substitution error	Pass
TC-05	NMT en→am	"Where is the clinic?"	Correct Amharic translation	Verified by native speaker	Pass
TC-06	NMT am→en	Amharic greeting	Correct English translation	Verified correct	Pass
TC-07	TTS — English	"The clinic is nearby."	Intelligible English audio	Confirmed by 3 listeners	Pass
TC-08	TTS — Amharic	Short Amharic sentence	Intelligible Amharic audio	Intelligible with minor accent	Pass
TC-09	Response time	15-word English sentence	< 5 seconds	Mean: 3.8 s across 10 trials	Pass
TC-10	Bidirectional	5-exchange conversation	Both directions work	All 5 exchanges completed	Pass

TC ID	Feature Tested	Input	Expected Result	Actual Result	Pass/Fail
TC-11	Offline mode	WiFi disabled	Pipeline executes offline	Executed; TTS used espeak-ng	Pass
TC-12	Noise tolerance (15 dB SNR)	Speech with fan noise	WER < 30%	WER ≈ 22%	Pass
TC-13	2-hour continuous operation	Continuous use 2 hours	No crashes	No crashes observed	Pass
TC-14	Short input rejection	0.3-second button press	Input ignored	Ignored correctly	Pass
TC-15	Audio privacy	Complete cycle	No audio files remain	Files deleted from tmpfs	Pass

6.3 Performance Test Results

Metric	Target	Measured Result
End-to-end response time (mean)	< 5 seconds	3.8 s (quiet), 4.3 s (noisy)
Word error rate — English	< 20%	8% in quiet environment
Word error rate — Amharic	< 20%	18% in quiet environment
TTS intelligibility (English)	Word intelligibility > 80%	95% (gTTS online)
TTS intelligibility (Amharic)	Word intelligibility > 80%	82% (gTTS), ~65% (espeak fallback)
Battery life	≥ 4 hours	4.5 hours from 10,000 mAh bank
System uptime (2 hours)	No crashes	No crashes in 3 separate sessions

6.4 Defects Identified and Resolved

Defect	Severity	Resolution
GPIO button debounce caused double-triggering	High	200 ms software debounce added in GPIO callback
Amharic TTS quality drops in offline mode (espeak-ng)	Medium	Documented as known limitation; online mode recommended
ASR fails on utterances longer than ~20 seconds	Low	Maximum recording duration of 20 seconds with truncation added

Defect	Severity	Resolution
NMT response time increases for complex multi-clause sentences	Medium	Sentence splitting added for inputs > 30 words

6.5 Risk and Contingency Plan

Risk	Impact	Contingency Plan
Background noise degrades ASR accuracy	High	Use directional microphones; recommend quiet environments
Online API unavailable during demonstration	Medium	Pre-test offline model fallback; cache demonstration sentences
Hardware failure during demonstration	High	Maintain spare hardware set; test all components before event
Power bank depletion	Medium	Charge to 100% before demonstration; bring USB-C supply

CHAPTER SEVEN: CONCLUSION AND RECOMMENDATION

7.1 Summary

This project designed, developed, and evaluated a portable two-user real-time speech translation device with button-controlled interaction. The device integrates a cascaded ASR-NMT-TTS pipeline on a Raspberry Pi 4 embedded platform, supporting bidirectional translation between Amharic and English — a language pair of direct relevance to the Ethiopian communication context.

Key achievements of the project include:

A complete end-to-end speech translation pipeline implemented and demonstrated on Raspberry Pi hardware.

Button-controlled hardware interaction successfully implemented using GPIO interrupts.

Partial offline operation achieved using locally deployed Whisper-tiny ASR and Helsinki-NLP Opus-MT models.

Mean end-to-end response time of 3.8 seconds in quiet conditions, within the 5-second target.

Amharic ASR WER of 18% and English ASR WER of 8% in controlled conditions.

Two-hour continuous operation verified without system failure.

The project demonstrated that low-cost Raspberry Pi hardware, open-source AI models, and a button-controlled interface can provide a functional and practically useful speech translation solution for two-person multilingual communication.

7.2 Limitations

Amharic TTS quality in offline mode (espeak-ng) is noticeably lower than online gTTS.

ASR accuracy for Amharic (18% WER) is higher than for English (8% WER), reflecting the relative scarcity of Amharic training data.

The system currently supports only the Amharic-English language pair.

Performance in high-noise environments (SNR < 10 dB) is expected to degrade significantly and was not tested.

7.3 Recommendations

Fine-tune the Whisper ASR model on a curated Amharic conversational speech dataset to reduce the word error rate below 10%.

Integrate a locally deployable Amharic TTS model to improve offline TTS quality.

Expand language support to include Afaan Oromoo, Tigrinya, and Somali using the same pipeline architecture with replacement model files.

Conduct field testing in actual deployment environments — health centers, government offices, and markets — to evaluate real-world performance.

Add a small OLED display to show recognized and translated text, reducing user uncertainty during processing.

Implement battery level monitoring and automatic low-power idle mode.

Package the device in a ruggedized enclosure suitable for outdoor use.

7.4 Conclusion

This project demonstrated that a portable, affordable, and partially offline-capable two-user real-time speech translation device can be successfully implemented using open-source AI

technologies and low-cost embedded hardware. The resulting prototype provides a practical communication tool for multilingual face-to-face interaction with direct applicability in healthcare, public service, education, and trade environments in Ethiopia and similar multilingual developing-country contexts.

The project contributes to the body of knowledge on edge AI deployment for low-resource language speech processing and provides a replicable hardware-software architecture that can serve as a foundation for future work on affordable multilingual communication devices.

REFERENCES

[1] Abate, S. T., Menzel, W., & Tafila, B. (2020). An Amharic speech corpus for large vocabulary continuous speech recognition. In Proceedings of LREC.

[2] Baevski, A., Zhou, H., Mohamed, A., & Auli, M. (2020). wav2vec 2.0: A framework for self-supervised learning of speech representations. NeurIPS 2020.

[3] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. Proceedings of ICLR 2015.

[4] Bansal, R., Kumar, A., & Singh, P. (2021). Real-time speech translation on embedded systems using edge AI. International Journal of Embedded Systems, 13(4), 312–325.

[5] Gaye, L., Holmquist, L. E., & Maze, R. (2019). Button interaction design for non-technical users in low-resource settings. Proceedings of CHI 2019.

[6] Gezmu, A. M., Nürnberger, A., & Schwarzkopf, J. (2021). Extended Amharic-English parallel corpus for machine translation. Proceedings of IWSLT 2021.

[7] Google. (2023). Google Translate. https://translate.google.com

[8] Graves, A., Mohamed, A., & Hinton, G. (2013). Speech recognition with deep recurrent neural networks. Proceedings of ICASSP 2013.

[9] Hinton, G., et al. (2012). Deep neural networks for acoustic modeling in speech recognition. IEEE Signal Processing Magazine, 29(6), 82–97.

[10] ITU. (2023). Measuring digital development: Facts and figures 2023. International Telecommunication Union.

[11] Pocketalk. (2023). Pocketalk S2 translation device. https://pocketalk.com

[12] Radford, A., et al. (2022). Robust speech recognition via large-scale weak supervision. arXiv:2212.04356.

[13] Raspberry Pi Foundation. (2023). Raspberry Pi 4 Model B product brief. https://www.raspberrypi.com

[14] Ren, Y., et al. (2021). FastSpeech 2: Fast and high-quality end-to-end text to speech. Proceedings of ICLR 2021.

[15] Shen, J., et al. (2018). Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions. Proceedings of ICASSP 2018.

[16] TensorFlow Lite. (2023). TFLite performance benchmarks. https://www.tensorflow.org/lite/performance/benchmarks

[17] Tiedemann, J., & Thottingal, S. (2020). OPUS-MT: Building open translation services for the world. Proceedings of EAMT 2020.

[18] van den Oord, A., et al. (2016). WaveNet: A generative model for raw audio. arXiv:1609.03499.

[19] Vaswani, A., et al. (2017). Attention is all you need. NeurIPS 2017.